Part1-MySQL

Task1：设计书籍信息数据库

```
mysql> show databases;
+--------------------+
| Database           |
+--------------------+
| information_schema |
| library_manage     |
| mysql              |
| performance_schema |
| sys                |
+--------------------+
```

```
mysql> select * from book;
+---------+--------------------+-----------------+-------+
| book_id | name               | author          | price |
+---------+--------------------+-----------------+-------+
|       1 | 十日终焉           | 杀虫队队员      |    60 |
|       2 | 传说管理局         | 杀虫队队员      |    30 |
|       3 | 全知读者视角       | 杨可意          |   100 |
+---------+--------------------+-----------------+-------+
3 rows in set (0.01 sec)
```

Task2：完成基本数据操作

```
mysql> select * from book
    -> ;
+---------+--------------------+-----------------+-------+
| book_id | name               | author          | price |
+---------+--------------------+-----------------+-------+
|       1 | 十日终焉           | 杀虫队队员      |    60 |
|       2 | 传说管理局         | 杀虫队队员      |    30 |
|       3 | 全知读者视角       | 杨可意          |   100 |
+---------+--------------------+-----------------+-------+
3 rows in set (0.01 sec)

mysql> select * from book where book_id = 1;
+---------+--------------+-----------------+-------+
| book_id | name         | author          | price |
+---------+--------------+-----------------+-------+
|       1 | 十日终焉     | 杀虫队队员      |    60 |
+---------+--------------+-----------------+-------+
1 row in set (0.00 sec)

mysql> UPDATE book SET price = 92.00 where book_id = 1;
Query OK, 1 row affected (0.02 sec)
Rows matched: 1  Changed: 1  Warnings: 0

mysql> select * from book where book_id = 1;
+---------+--------------+-----------------+-------+
| book_id | name         | author          | price |
+---------+--------------+-----------------+-------+
|       1 | 十日终焉     | 杀虫队队员      |    92 |
+---------+--------------+-----------------+-------+
1 row in set (0.00 sec)

mysql> delete from book where book_id = 3;
Query OK, 1 row affected (0.01 sec)

mysql> select * from book;
+---------+-----------------+-----------------+-------+
| book_id | name            | author          | price |
+---------+-----------------+-----------------+-------+
|       1 | 十日终焉        | 杀虫队队员      |    92 |
|       2 | 传说管理局      | 杀虫队队员      |    30 |
+---------+-----------------+-----------------+-------+
2 rows in set (0.00 sec)
```

Task3：理解数据库设计

1. 你是否选择了某个字段作为唯一标识？为什么选择该字段作为唯一标识？

- 我选了**book_id**作为唯一标识,因为book_id我设置为了**主键**,主键不可以重复,并且我设置**AUTO_INCREMENT**保证了加入新数据不用手动设置,并且可以**自增**

2. 如果存在两本书籍名称相同的情况，数据库应该如何区分这两条记录？

- 依靠book_id区分,因为设置为**主键**,具有唯一性

3. 对MySQL的理解:

- 优点:
  - 比较轻量化,部署很方便
  - 学习SQL语法很方便
  - 速度算比较快的,并且开源免费
- 缺点:
  - 对大量数据,高并发场景性能下降

Part2-Docker

1. 拉取mysql:8.0

```
(base) sterben@wzy:~$ docker pull mysql:8.0
8.0: Pulling from library/mysql
96d30d9fbee8: Pull complete
49ec2dab01d9: Pull complete
0d74d296605b: Pull complete
297d04cfe470: Pull complete
e3e5d1ac74c1: Pull complete
a63160a5eda1: Pull complete
7534d1db9f8d: Pull complete
6ef6c7b50a93: Pull complete
4c8a3e0d4e4b: Pull complete
ab24264a27e9: Pull complete
edf85873f64e: Pull complete
796812c73292: Download complete
af166387641d: Download complete
Digest: sha256:7dcddc01f13bab2f15cde676d44d01f61fc9f99fe7785e86196dfc07d358ae2b
Status: Downloaded newer image for mysql:8.0
docker.io/library/mysql:8.0
```

2. 创建容器

```
(base) sterben@wzy:~$ docker run -d \
  --name mysql-user-db \
  -p 3306:3306 \
  -v ~/mysql_data:/var/lib/mysql \
  -e MYSQL_ROOT_PASSWORD=123456 \
  -e MYSQL_DATABASE=user_db \
  mysql:8.0
f27e3d234ef64d499551e78f62fa26c04f8da357f49f8b5c30382e6d346822e0
docker: Error response from daemon: failed to set up container networking: driver failed programming external connectivi
ty on endpoint mysql-user-db (8e49631284a33ef2fdc14eee0aa81e35b03c48902413dcf785a3909e07a84814): failed to bind host por
t 0.0.0.0:3306/tcp: address already in use
```

这里遇到了一个问题,我之前下载的MySQL已经**占用了宿主机的3306**,所以我这里删除了先前创建的容器,并且修改为了`-p 3307:3306`将**宿主机的3307端口映射到容器的3306端口**

```
(base) sterben@wzy:~$ docker rm -f mysql-user-db
mysql-user-db
(base) sterben@wzy:~$ docker run -d \
  --name mysql-user-db \
  -p 3307:3306 \
  -v ~/mysql_data:/var/lib/mysql \
  -e MYSQL_ROOT_PASSWORD=123456 \
  -e MYSQL_DATABASE=user_db \
  mysql:8.0
96bc2e53a6f36b77cdf81ddd1de0758d0c0cdd05ae886bf9d02263cbe311dd31
```

3. 进入mysql-user-db中的Mysql

```
(base) sterben@wzy:~$ docker exec -it mysql-user-db bash
bash-5.1# mysql -uroot -p123456
mysql: [Warning] Using a password on the command line interface can be insecure.
Welcome to the MySQL monitor.  Commands end with ; or \g.
Your MySQL connection id is 8
Server version: 8.0.46 MySQL Community Server - GPL

Copyright (c) 2000, 2026, Oracle and/or its affiliates.

Oracle is a registered trademark of Oracle Corporation and/or its
affiliates. Other names may be trademarks of their respective
owners.

Type 'help;' or '\h' for help. Type '\c' to clear the current input statement.

mysql>
```

4. 这里又遇到了一个问题:在mysql中创建表的时候我发现key不能以中文命名,所以我退出了容器并修改环境为utf-8

```
mysql> exit
Bye
bash-5.1# exit
exit
(base) sterben@wzy:~$ docker exec -it mysql-user-db env LANG=C.UTF-8 /bin/bash
```

5. 创建表并插入数据

```
mysql> show tables;
+--------------------+
| Tables_in_user_db |
+--------------------+
| user               |
+--------------------+
1 row in set (0.01 sec)

mysql> select * from user;
+----+--------+
| id | name   |
+----+--------+
|  1 | 张三   |
|  2 | 李四   |
|  3 | 王五   |
+----+--------+
3 rows in set (0.01 sec)
```

6. 删除容器

```
bash-5.1# exit
exit
Terminal close -- exit!
(base) sterben@wzy:~$ docker rm -f mysql-user-db
mysql-user-db
```

7. 重新拉起容器并验证表是否存在

```
(base) sterben@wzy:~$ docker rm -f mysql-user-db
mysql-user-db
mysql> use user_db;
Reading table information fo
You can turn off this featur

Database changed
mysql> show tables;
+------------------+
| Tables_in_user_db |
+------------------+
| user             |
+------------------+
1 row in set (0.00 sec)

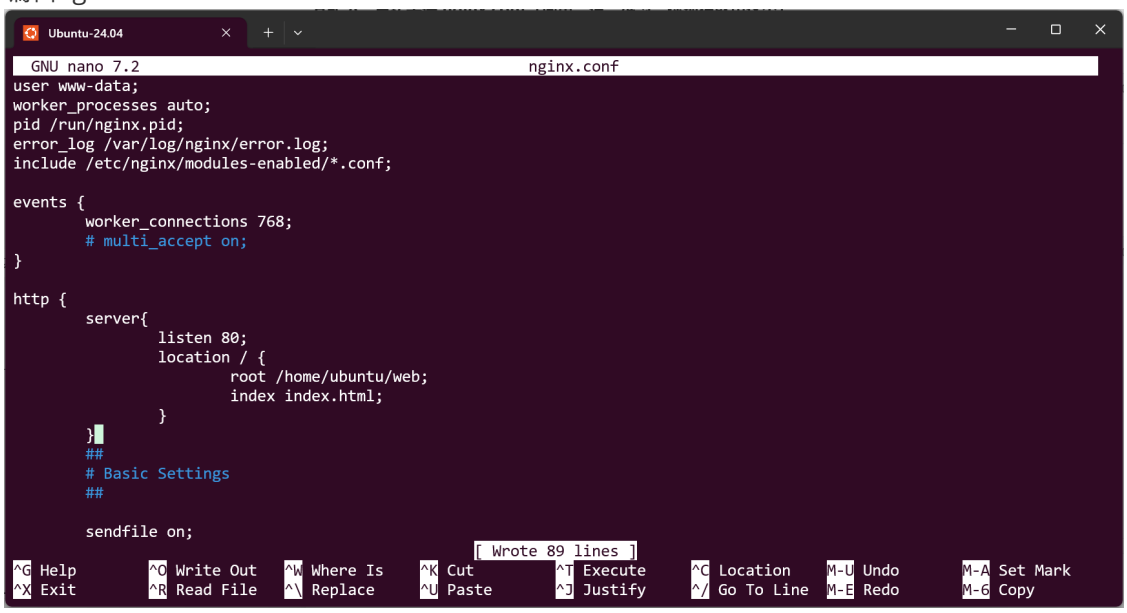
mysql> select*from user;
+----+--------+
| id | name   |
+----+--------+
|  1 | 张三   |
|  2 | 李四   |
|  3 | 王五   |
+----+--------+
3 rows in set (0.00 sec)
```

8. 如果我是出题人,我会问:

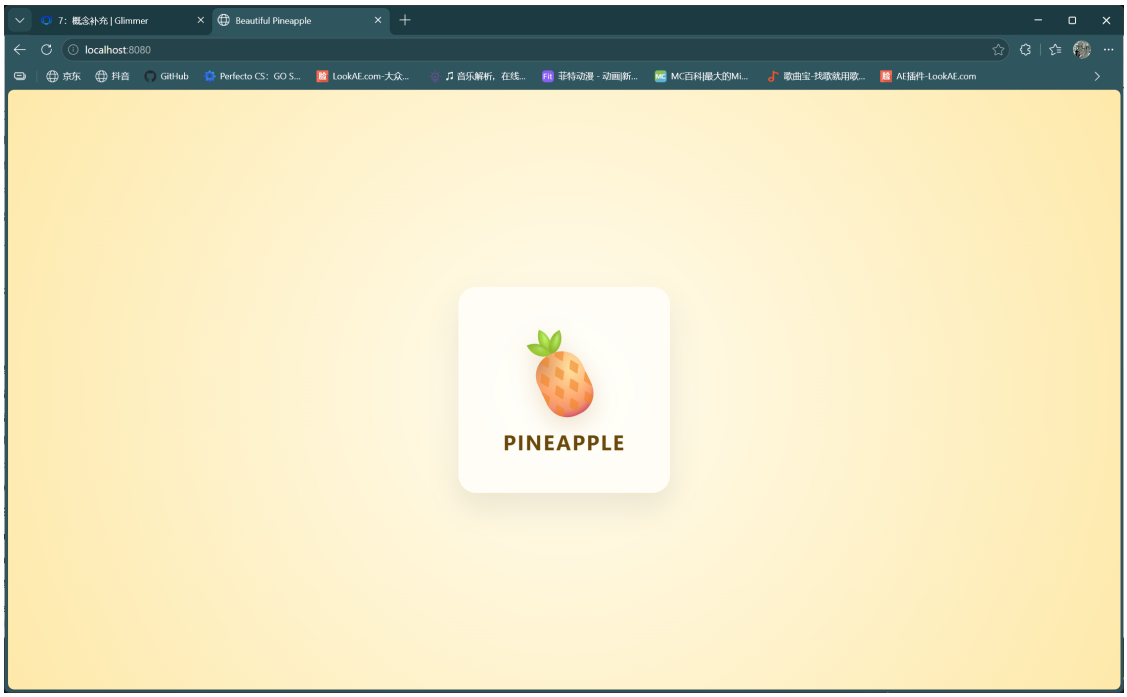
- `docker exec -it` 中 `-i` 和 `-t` 分别干什么?
  答:-i 保持标准输入; -t 分配伪终端;
- 为什么可能在映射宿主的3306端口时会报错,并看懂报错信息并解决

Part3-Nginx

1. 编辑nginx.conf



2. 访问网页



Part4-HTTP

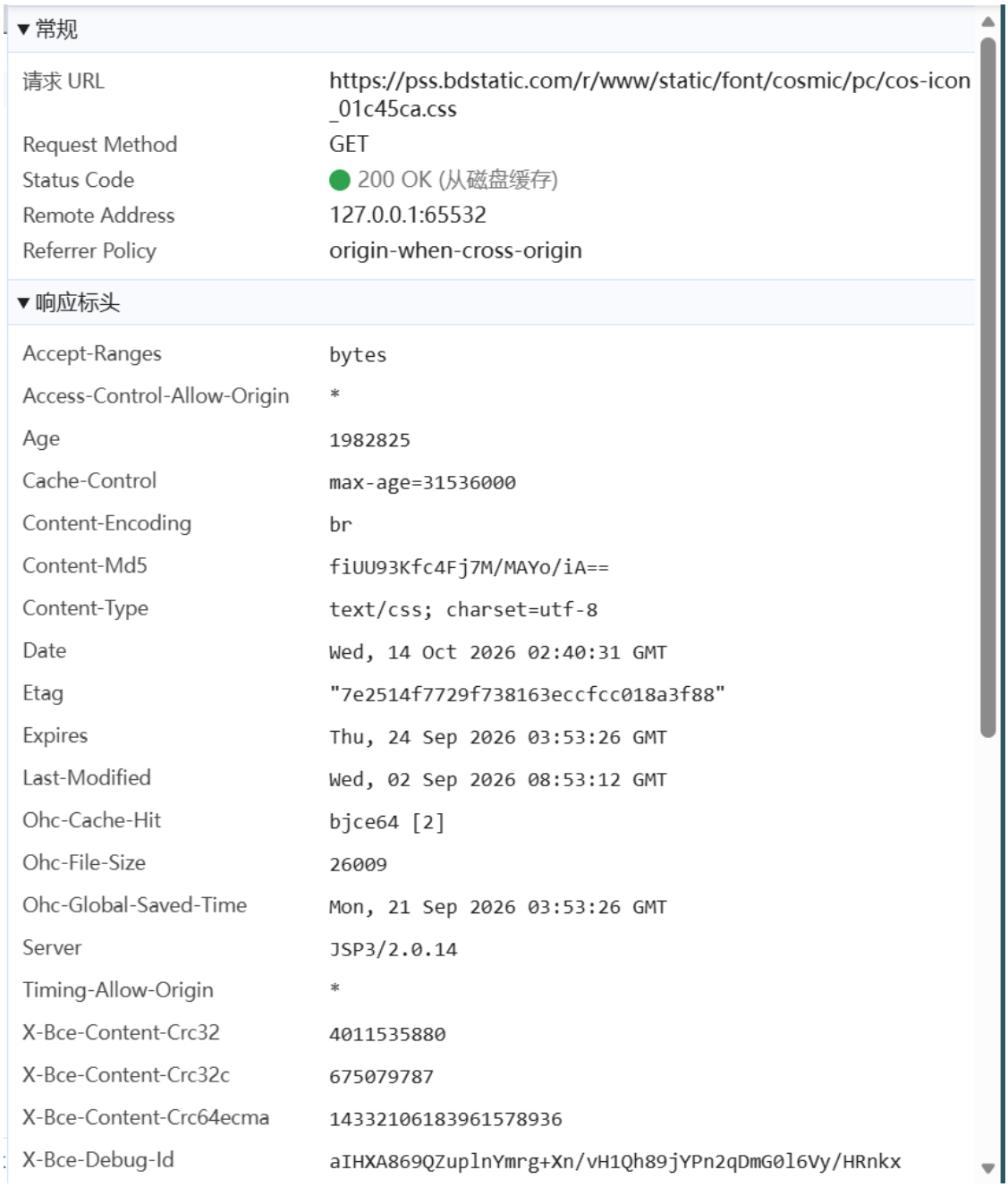
1. HTTP 是什么？解决了什么问题？

- HTTP是客户端和服务器之间通信的规则
- 定义请求长什么样、响应长什么样，客户端和服务器不用关心对方用什么语言、什么操作系统，统一一套格式就能通信。

2. 请你对常见的状态码进行了解并进行总结

- 1xx：信息
- 2xx：成功
- 3xx：重定向
- 4xx：客户端错误
- 5xx：服务端错误

3. 随意选取了一条请求



▼常规

请求 URL	https://pss.bdstatic.com/r/www/static/font/cosmic/pc/cos-icon_01c45ca.css
Request Method	GET
Status Code	200 OK (从磁盘缓存)
Remote Address	127.0.0.1:65532
Referrer Policy	origin-when-cross-origin

▼响应标头

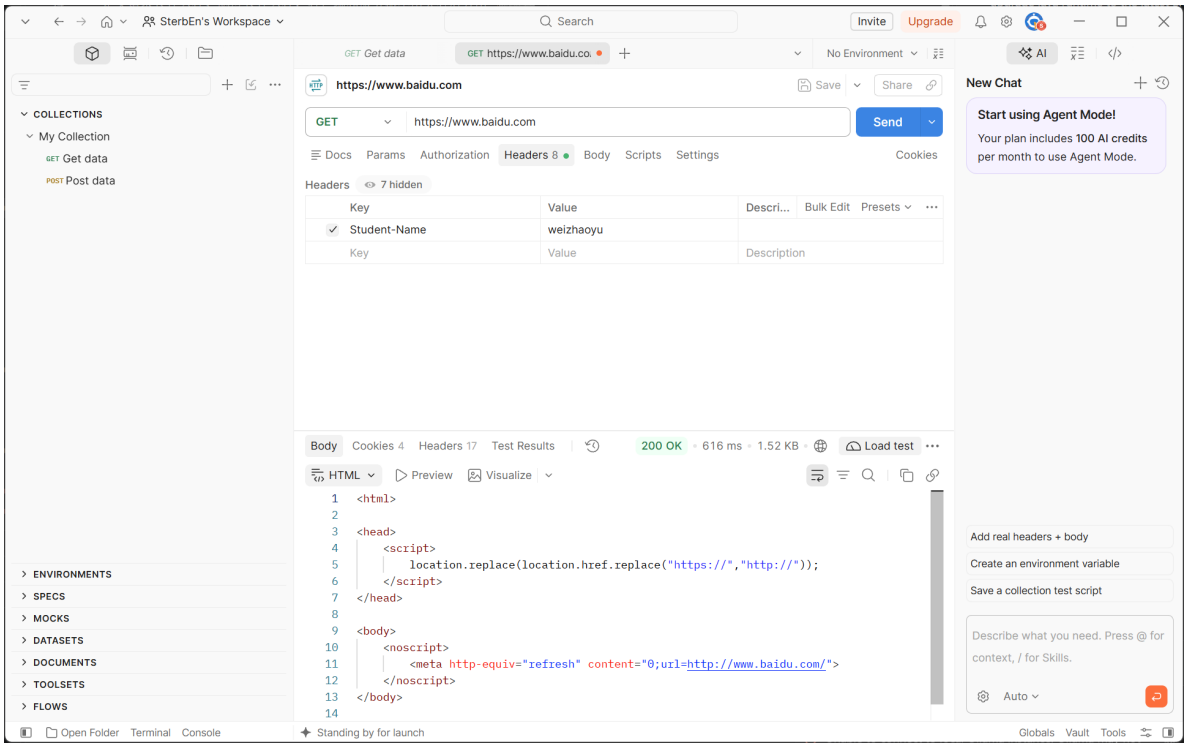
Accept-Ranges	bytes
Access-Control-Allow-Origin	*
Age	1982825
Cache-Control	max-age=31536000
Content-Encoding	br
Content-Md5	fiUU93Kfc4Fj7M/MAYo/iA==
Content-Type	text/css; charset=utf-8
Date	Wed, 14 Oct 2026 02:40:31 GMT
Etag	"7e2514f7729f738163eccfcc018a3f88"
Expires	Thu, 24 Sep 2026 03:53:26 GMT
Last-Modified	Wed, 02 Sep 2026 08:53:12 GMT
Ohc-Cache-Hit	bjce64 [2]
Ohc-File-Size	26009
Ohc-Global-Saved-Time	Mon, 21 Sep 2026 03:53:26 GMT
Server	JSP3/2.0.14
Timing-Allow-Origin	*
X-Bce-Content-Crc32	4011535880
X-Bce-Content-Crc32c	675079787
X-Bce-Content-Crc64ecma	14332106183961578936
X-Bce-Debug-Id	aIHXA869QZuplnYmrg+Xn/vH1Qh89jYPn2qDmG0l6Vy/HRnkx

- 方法是GET
- header如图所示

4. 一次 HTTP 请求通常包含哪些部分？分别有什么作用?
    1. 请求行:告诉服务器用什么**方法**,访问什么**资源**,使用哪个版本的**HTTP协议**
    2. 请求头:附加说明信息,比如用户身份和数据格式之类
    3. 请求体:一般POST请求才有,一般存放业务数据
5. 常见 HTTP 请求方法
    1. GET:只读,获取服务器中的数据
       场景:查询用户信息
    2. POST:新建资源,提交数据给服务器
       场景:新增用户,新增订单
    3. PUT:全局变量替换,覆盖旧数据
       场景:完整修改用户信息
    4. PATCH:局部修改变量,只传需要修改的字段
       场景:修改用户部分数据

5. DELETE:删除资源
   场景:删除用户的id,手机号

Task2：修改请求 Header



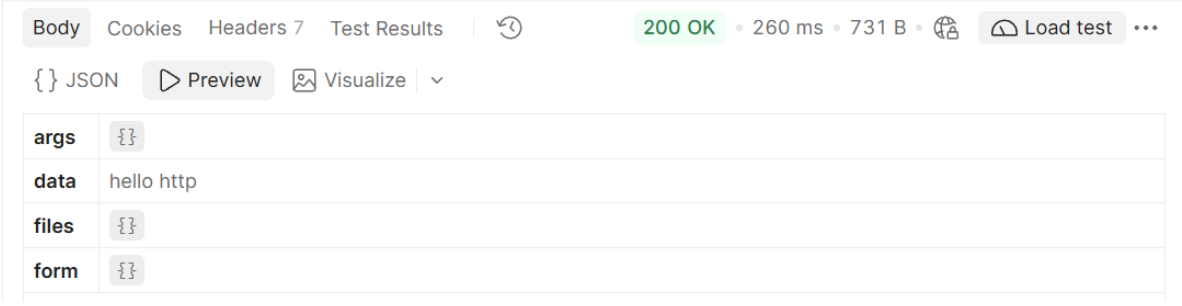
1. 为什么 HTTP 请求需要 Header?

- **请求行包含信息太少**,header要告诉服务器如何解析本次请求

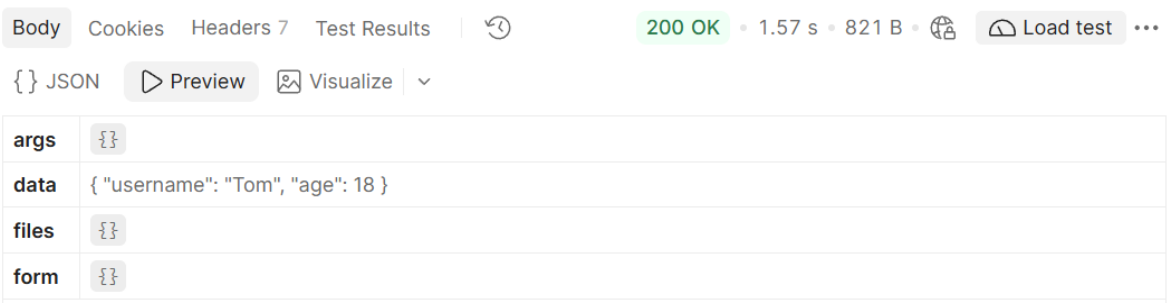
2. 如果 HTTP 请求没有 Header，服务器还能正常处理所有请求吗？有没有例外情况?

- 大多数情况不行
- 例外:简单的GET请求可以处理,仅有请求行,有可能成功

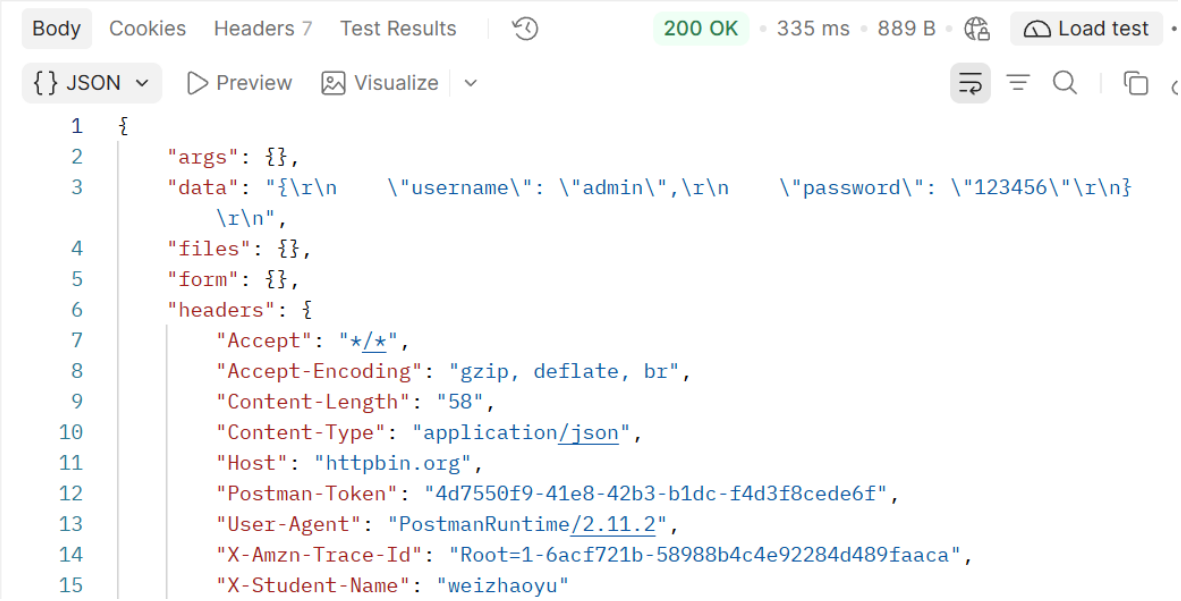
Task3-1：发送普通文本



Task3-2：发送 JSON 数据



Task4：模拟一个简单登录请求



1. 为什么登录请求通常使用 POST，而不是 GET?

- GET是只读,而登录是**提交凭证**进行校验,而且GET会把用户名和密码拼在URL地址上

2. 为什么需要设置 Content-Type: application/json?

- 让服务器知道这段文本是json格式,用json解析器

3. 用户名和密码是否应该直接放在 URL 中？为什么?

- 不应该,因为这样极易泄露,一眼就能看出密码